

Lexical Analysis – Part 2: Converting Regular Expression to DFA

1. Introduction to Direct RE to DFA Conversion

1.1 Overview

Main Concept: We can convert a Regular Expression (RE) directly to a DFA without first creating an NFA.

Key Advantage: This method is more efficient than the traditional RE → NFA → DFA approach.

Process Summary:

STEP 1: Augment the regular expression

STEP 2: Create a syntax tree

STEP 3: Number the positions (leaves)

STEP 4: Calculate nullable, firstpos, lastpos for all nodes

STEP 5: Calculate followpos for all positions

STEP 6: Construct the DFA using followpos

2. Augmentation of Regular Expression

2.1 What is Augmentation?

Definition: Augmentation means adding a special end marker symbol # to the regular expression.

Original RE: r

Augmented RE: (r)#

Purpose of #:

- Marks the end of the string
- Helps identify accepting states
- Makes pattern matching unambiguous

2.2 Examples of Augmentation

Example 1:

Original RE: a|b

Augmented RE: (a|b)#

Example 2:

Original RE: $(a|b)^*a$

Augmented RE: $((a|b)^*a)\#$

Example 3:

Original RE: ab^*c

Augmented RE: $(ab^*c)\#$

Example 4:

Original RE: $(a|\epsilon)bc^*$

Augmented RE: $((a|\epsilon)bc^*)\#$

3. Syntax Tree Construction

3.1 What is a Syntax Tree?

A syntax tree is a tree representation of the augmented regular expression where:

LEAVES: Contain alphabet symbols, #, and ϵ (empty string)

INTERNAL NODES: Contain operators ($|$, $*$, $\cdot$ for concatenation)

ROOT: Represents the entire expression

3.2 Position Numbering

Each alphabet symbol and # in the syntax tree is assigned a unique **position number**.

Important Notes:

- Numbers start from 1
- Each symbol gets exactly one number
- ϵ (epsilon) is NOT numbered
- Operator nodes are NOT numbered

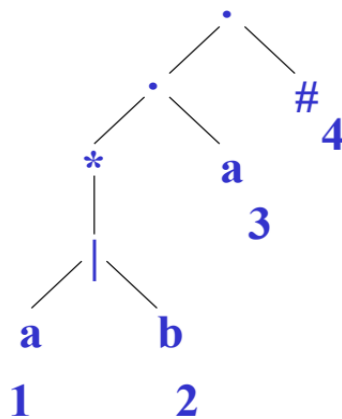
3.3 Example: Syntax Tree for $(a|b)^*a\#$

Step-by-Step Construction:

Original RE: $(a|b)^*a$

Augmented RE: $((a|b)^*a)\#$

Syntax Tree:



4. Important Functions: nullable, firstpos, lastpos

4.1 Definitions

Function 1: nullable(n)

Definition: nullable(n) is true for a syntax tree node n if and only if the regular expression represented by n has ϵ in its language.

Simple Meaning: Can this sub-expression match nothing?

First, what does nullable(n) mean?

nullable(n) tells us whether a node n can produce the empty string (ϵ).

- true $\rightarrow$ it **can** produce ϵ
- false $\rightarrow$ it **cannot** produce ϵ

What is ϵ (epsilon)?

- ϵ means **empty string**
- It has **length 0**
- Example:
 - Language $\{ \epsilon \} \rightarrow$ contains only the empty string
 - Language $\{ "a", "ab" \} \rightarrow$ does **not** contain ϵ

What is “the regular expression represented by node n”?

In a **syntax tree of a regular expression**,

- each node represents **part of the regular expression**
- n = one such node

So: "the regular expression represented by n " means "the sub-expression at that node"

Now the sentence in very simple words, $\text{nullable}(n)$ is true if and only if the sub-expression at node n can generate the empty string (ϵ). That's it.

Examples (this makes it clear)

1 Simple symbol

Regular expression: a

- Language = $\{ "a" \}$
- Does it contain ϵ ?  So, $\text{nullable}(a) = \text{false}$

2 Empty string

Regular expression: ϵ

- Language = $\{ \epsilon \}$
- Contains ϵ ?  So, $\text{nullable}(\epsilon) = \text{true}$

3 Kleene star

Regular expression: a^*

- Language = $\{ \epsilon, "a", "aa", "aaa", \dots \}$
- Contains ϵ ?  So, $\text{nullable}(a^*) = \text{true}$

4 Concatenation

Regular expression: ab

- Language = $\{ "ab" \}$
- Contains ϵ ?  So, $\text{nullable}(ab) = \text{false}$

5 Union (OR)

Regular expression: $a \mid \epsilon$

- Language = $\{ "a", \epsilon \}$
- Contains ϵ ?  So, $\text{nullable}(a \mid \epsilon) = \text{true}$

Function 2: $\text{firstpos}(n)$

Definition: The set of positions of symbols that can appear FIRST in strings generated by the sub-expression rooted at node n

Simple Meaning: Which positions can start a string?

Function 3: lastpos(n)

Definition: The set of positions of symbols that can appear LAST in strings generated by the sub-expression rooted at node n

Simple Meaning: Which positions can end a string?

4.2 Computation Rules Table

COMPLETE RULES TABLE:

Node n	nullable(n)	firstpos(n)	lastpos(n)
n is a leaf node labeled ϵ	true	$\emptyset$	$\emptyset$
n is a leaf node labelled with position i	false	{ i }	{ i }
n is an or node with left child c1 and right child c2	nullable(c1) or nullable(c2)	firstpos(c1) $\cup$ firstpos(c2)	lastpos(c1) $\cup$ lastpos(c2)
n is a cat node with left child c1 and right child c2	nullable(c1) and nullable(c2)	If nullable(c1) then firstpos(c1) $\cup$ firstpos(c2) else firstpos(c1)	If nullable(c2) then lastpos(c2) $\cup$ lastpos(c1) else lastpos(c2)
n is a star node with child node c1	true	firstpos(c1)	lastpos(c1)

4.3 Detailed Explanation of Each Rule

RULE 1: Leaf labeled ϵ

Example: Node containing ϵ

nullable(n) = TRUE

Reason: ϵ represents empty string, so it IS nullable

firstpos(n) = $\emptyset$

Reason: Empty string has no first position

lastpos(n) = $\emptyset$

Reason: Empty string has no last position

RULE 2: Leaf labeled with symbol at position i

Example: Node containing 'a' at position 5

$\text{nullable}(n) = \text{FALSE}$

Reason: A single symbol is NOT empty string

$\text{firstpos}(n) = \{5\}$

Reason: The symbol at position 5 is the first (and only) position

$\text{lastpos}(n) = \{5\}$

Reason: The symbol at position 5 is the last (and only) position

RULE 3: OR node ($c_1 \mid c_2$)

Example: ($a \mid b$) where a is position 1, b is position 2

$\text{nullable}(n) = \text{nullable}(c_1) \text{ OR } \text{nullable}(c_2)$

Reason: Expression is nullable if EITHER branch can be empty

Example: If c_1 can generate ϵ , then $(c_1 \mid c_2)$ can generate ϵ

$\text{firstpos}(n) = \text{firstpos}(c_1) \cup \text{firstpos}(c_2)$

Reason: First position can come from EITHER branch

Example: First symbol can be from c_1 OR from c_2

$\text{lastpos}(n) = \text{lastpos}(c_1) \cup \text{lastpos}(c_2)$

Reason: Last position can come from EITHER branch

Example: Last symbol can be from c_1 OR from c_2

RULE 4: CONCATENATION node ($c_1 \cdot c_2$)

Example: ab where a is position 1, b is position 2

$\text{nullable}(n) = \text{nullable}(c_1) \text{ AND } \text{nullable}(c_2)$

Reason: BOTH parts must be nullable for concatenation to be nullable

Example: ab is nullable only if both a and b can be ϵ

$\text{firstpos}(n)$:

IF $\text{nullable}(c_1)$ is TRUE:

$\text{firstpos}(n) = \text{firstpos}(c_2)$

Reason: If c_1 can be empty, first symbol might come from c_2

ELSE:

$\text{firstpos}(n) = \text{firstpos}(c_1)$

Reason: If c_1 cannot be empty, first symbol must come from c_1

```

lastpos(n):
  IF nullable(c2) is TRUE:
    lastpos(n) = lastpos(c1) U lastpos(c2)
    Reason: If c2 can be empty, last symbol might come from c1
  ELSE:
    lastpos(n) = lastpos(c2)
    Reason: If c2 cannot be empty, last symbol must come from c2

```

RULE 5: STAR node (c₁)*

Example: a* where a is position 1

nullable(n) = TRUE

Reason: Star always allows zero occurrences (matches ϵ)

firstpos(n) = firstpos(c₁)

Reason: If string is non-empty, first symbol comes from c₁

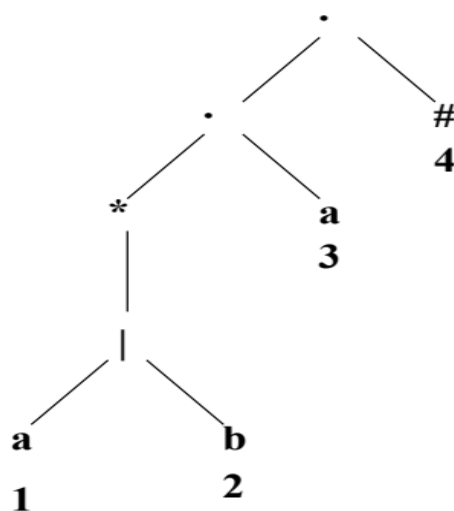
lastpos(n) = lastpos(c₁)

Reason: If string is non-empty, last symbol comes from c₁

5. Complete Example: Computing nullable, firstpos, lastpos

5.1 Example: (a|b)*a#

Syntax Tree with Position Numbers:



5.2 Bottom-Up Calculation

STEP 1: Calculate for leaves

Node: a at position 1

nullable = FALSE

firstpos = {1}

lastpos = {1}

Node: b at position 2

nullable = FALSE

firstpos = {2}

lastpos = {2}

Node: a at position 3

nullable = FALSE

firstpos = {3}

lastpos = {3}

Node: # at position 4

nullable = FALSE

firstpos = {4}

lastpos = {4}

STEP 2: Calculate for OR node (a|b)

Node: (a|b) where a=position 1, b=position 2

nullable = nullable(a) OR nullable(b)

= FALSE OR FALSE = FALSE

firstpos = firstpos(a) $\cup$ firstpos(b)

= {1} $\cup$ {2} = {1, 2}

lastpos = lastpos(a) $\cup$ lastpos(b)

= {1} $\cup$ {2} = {1, 2}

STEP 3: Calculate for STAR node $(a \mid b)^*$

Node: $(a \mid b)^*$

nullable = TRUE (star is always nullable)

firstpos = firstpos($a \mid b$)

= {1, 2}

lastpos = lastpos($a \mid b$)

= {1, 2}

STEP 4: CONCATENATION ($(a \mid b)^* \cdot a$)

Let:

- $c_1 = (a \mid b)^*$
- $c_2 = a(3)$

nullable

nullable = TRUE AND FALSE = FALSE

firstpos

Rule:

- If nullable(c_1) = TRUE $\rightarrow$ union

firstpos = firstpos(c_1) $\cup$ firstpos(c_2)

= {1,2} $\cup$ {3}

= {1,2,3}

lastpos

Rule:

- If nullable(c_2) = FALSE $\rightarrow$ take lastpos(c_2)

lastpos = lastpos(a)

= {3}

STEP 5: ROOT CONCATENATION ($(a \mid b)^* \cdot a \cdot \#$)

Let:

- $c_1 = ((a \mid b)^* \cdot a)$

- $c_2 = \#(4)$

nullable

nullable = FALSE AND FALSE = FALSE

firstpos

Rule:

- If nullable(c_1) = FALSE $\rightarrow$ take firstpos(c_1)

firstpos = {1,2,3}

lastpos

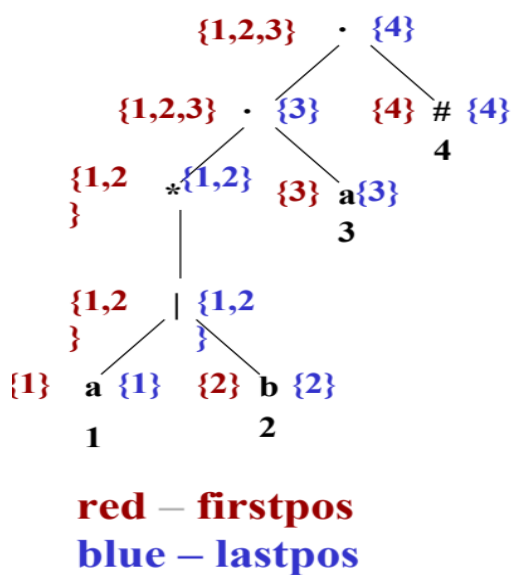
Rule:

- If nullable(c_2) = FALSE $\rightarrow$ take lastpos(c_2)

lastpos = {4}

5.3 Final Results Summary

Complete Tree with All Values:



6. followpos Function

6.1 Definition

followpos(i): The set of positions that can follow position i in strings generated by the augmented regular expression.

Simple Meaning: After reading the symbol at position i, which positions can come next?

6.2 Two Rules for Computing followpos

RULE 1: Concatenation Node Rule

If node n is a concatenation node $(\cdot)$ with:

- Left child: c_1
- Right child: c_2

AND position i is in $\text{lastpos}(c_1)$

THEN all positions in $\text{firstpos}(c_2)$ are added to $\text{followpos}(i)$

Why? Because in concatenation, what ends c_1 is followed by what starts c_2

Visual:

$$\begin{array}{c} n (\cdot) \\ / \backslash \\ c_1 \quad c_2 \end{array}$$

If $i \in \text{lastpos}(c_1)$, then $\text{followpos}(i)$ includes $\text{firstpos}(c_2)$

RULE 2: Star Node Rule

If node n is a star node $(*)$ with child c_1

AND position i is in $\text{lastpos}(n)$

THEN all positions in $\text{firstpos}(n)$ are added to $\text{followpos}(i)$

Why? Because under star, after the last symbol, we can loop back to first

Visual:

$$\begin{array}{c} n (*) \\ | \\ c_1 \end{array}$$

If $i \in \text{lastpos}(n)$, then $\text{followpos}(i)$ includes $\text{firstpos}(n)$

6.3 Algorithm to Compute followpos

STEP 1: Initialize $\text{followpos}(i) = \emptyset$ for all positions i

STEP 2: Do a depth-first traversal of the syntax tree

STEP 3: For each node n :

IF n is a concatenation node ($\cdot$):

c_1 = left child

c_2 = right child

FOR each position i in $\text{lastpos}(c_1)$:

$\text{followpos}(i) = \text{followpos}(i) \cup \text{firstpos}(c_2)$

IF n is a star node ($*$):

FOR each position i in $\text{lastpos}(n)$:

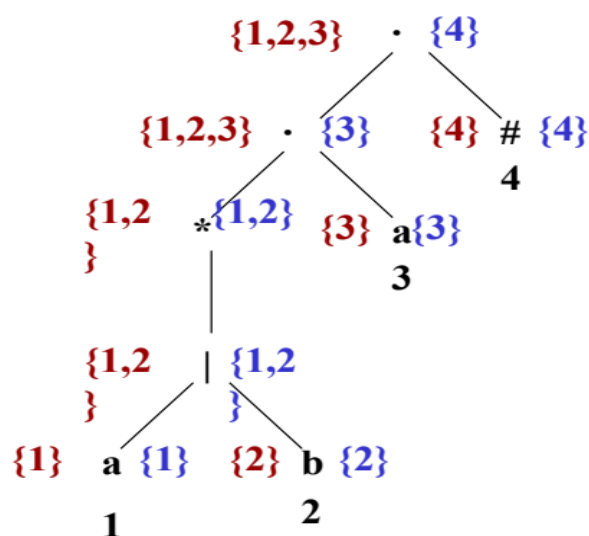
$\text{followpos}(i) = \text{followpos}(i) \cup \text{firstpos}(n)$

STEP 4: Return all followpos values

7. Complete Example: Computing followpos

7.1 Example: $(a|b)^*a\#$

Syntax Tree with firstpos and lastpos:



red – firstpos
blue – lastpos

$\text{followpos}(1) = \{1,2,3\}$

$\text{followpos}(2) = \{1,2,3\}$

$\text{followpos}(3) = \{4\}$

$\text{followpos}(4) = \{\}$

7.2 Step-by-Step followpos Computation

Initialize:

$$\text{followpos}(1) = \emptyset$$

$$\text{followpos}(2) = \emptyset$$

$$\text{followpos}(3) = \emptyset$$

$$\text{followpos}(4) = \emptyset$$

Apply STAR RULE at node $(a|b)^*$:

Node: $(a|b)^*$ is a star node

$$\text{lastpos}((a|b)^*) = \{1, 2\}$$

$$\text{firstpos}((a|b)^*) = \{1, 2\}$$

For each i in lastpos :

$$i = 1: \text{followpos}(1) = \text{followpos}(1) \cup \{1, 2\} = \{1, 2\}$$

$$i = 2: \text{followpos}(2) = \text{followpos}(2) \cup \{1, 2\} = \{1, 2\}$$

Apply CONCATENATION RULE at inner concatenation node $(\cdot)$ for $(a|b)^* \cdot a$:

Node: $(a|b) \cdot a$ is concatenation

Left child $c_1 = (a|b)$

Right child $c_2 = a$ at position 3

$$\text{lastpos}(c_1) = \{1, 2\}$$

$$\text{firstpos}(c_2) = \{3\}$$

For each i in $\text{lastpos}(c_1)$:

$$i = 1: \text{followpos}(1) = \text{followpos}(1) \cup \{3\} = \{1, 2\} \cup \{3\} = \{1, 2, 3\}$$

$$i = 2: \text{followpos}(2) = \text{followpos}(2) \cup \{3\} = \{1, 2\} \cup \{3\} = \{1, 2, 3\}$$

Apply CONCATENATION RULE at root node $(\cdot)$ for $((a|b)^* \cdot a) \cdot \#$:

Node: Root $\cdot$ is concatenation

Left child $c_1 = (a|b)^* \cdot a$

Right child $c_2 = \#$ at position 4

$$\text{lastpos}(c_1) = \{3\}$$

$$\text{firstpos}(c_2) = \{4\}$$

For each i in $\text{lastpos}(c_i)$:

$i = 3$: $\text{followpos}(3) = \text{followpos}(3) \cup \{4\} = \{4\}$

7.3 Final followpos Values

SUMMARY:

$\text{followpos}(1) = \{1, 2, 3\}$

$\text{followpos}(2) = \{1, 2, 3\}$

$\text{followpos}(3) = \{4\}$

$\text{followpos}(4) = \emptyset$

Interpretation:

- After position 1 (a), we can go to positions 1, 2, or 3
- After position 2 (b), we can go to positions 1, 2, or 3
- After position 3 (a), we must go to position 4 (#)
- After position 4 (#), nowhere to go (end of string)

8. Constructing DFA from followpos

8.1 DFA Construction Algorithm

ALGORITHM: Build DFA from followpos

INPUT:

- Augmented regular expression $(r)\#$
- followpos for all positions
- Symbol at each position

OUTPUT: DFA

STEP 1: Create start state

$S_1 = \text{firstpos}(\text{root})$

Mark S_1 as unmarked

This is the start state

STEP 2: WHILE there exists an unmarked state S:

STEP 2a: Mark state S

STEP 2b: FOR each input symbol a:

STEP 2b-i: Find all positions in S that have symbol a

Let these be $s_1, s_2, \dots, s_n$

STEP 2b-ii: Compute new state

$S' = \text{followpos}(s_1) \cup \text{followpos}(s_2) \cup \dots \cup \text{followpos}(s_n)$

STEP 2b-iii: Create transition

$\text{move}(S, a) = S'$

STEP 2b-iv: IF S' is not empty AND S' is not already in DFA states:

Add S' to DFA states as unmarked state

STEP 3: Mark accepting states

Any state containing position of # is an accepting state

STEP 4: Return DFA

8.2 Complete Example: $(a|b)^*a\#$

Given Information:

Positions and Symbols:

Position 1: a

Position 2: b

Position 3: a

Position 4: #

followpos values:

$\text{followpos}(1) = \{1, 2, 3\}$

$\text{followpos}(2) = \{1, 2, 3\}$

$\text{followpos}(3) = \{4\}$

$\text{followpos}(4) = \emptyset$

$\text{firstpos}(\text{root}) = \{1, 2, 3\}$

DFA Construction Steps:

ITERATION 1:

Start State: $S_1 = \text{firstpos}(\text{root}) = \{1, 2, 3\}$

Mark S_1 as unmarked

Process S_1 :

Mark S_1

For symbol 'a':

Positions in S_1 with symbol 'a': $\{1, 3\}$

$S' = \text{followpos}(1) \cup \text{followpos}(3)$

$= \{1, 2, 3\} \cup \{4\}$

$= \{1, 2, 3, 4\}$

Create state $S_2 = \{1, 2, 3, 4\}$

Transition: $\text{move}(S_1, a) = S_2$

S_2 is new, add as unmarked

For symbol 'b':

Positions in S_1 with symbol 'b': $\{2\}$

$S' = \text{followpos}(2)$

$= \{1, 2, 3\}$

$= S_1$ (already exists)

Transition: $\text{move}(S_1, b) = S_1$

After Iteration 1:

States: S_1 (marked), S_2 (unmarked)

Transitions:

$S_1 \xrightarrow{a} S_2$

$S_1 \xrightarrow{b} S_1$

ITERATION 2:

Process $S_2 = \{1, 2, 3, 4\}$:

Mark S_2

For symbol 'a':

Positions in S_2 with symbol 'a': $\{1, 3\}$

$S' = \text{followpos}(1) \cup \text{followpos}(3)$

$= \{1, 2, 3\} \cup \{4\}$

$= \{1, 2, 3, 4\}$

$= S_2$ (already exists)

Transition: $\text{move}(S_2, a) = S_2$

For symbol 'b':

Positions in S_2 with symbol 'b': $\{2\}$

$S' = \text{followpos}(2)$

$= \{1, 2, 3\}$

$= S_1$ (already exists)

Transition: $\text{move}(S_2, b) = S_1$

After Iteration 2:

States: S_1 (marked), S_2 (marked)

All states are marked - DONE

Complete Transitions:

$S_1 \xrightarrow{a} S_2$

$S_1 \xrightarrow{b} S_1$

$S_2 \xrightarrow{a} S_2$

$S_2 \xrightarrow{b} S_1$

STEP 3: Identify Accepting States

Accepting states = states containing position 4 (the # position)

$S_1 = \{1, 2, 3\}$ - does NOT contain 4 - NOT accepting

$S_2 = \{1, 2, 3, 4\}$ - CONTAINS 4 - ACCEPTING

Accepting states: $\{S_2\}$

8.3 Final DFA

Complete DFA:

States: $\{S_1, S_2\}$

Start State: S_1

Accepting States: $\{S_2\}$

Transitions:

$\delta(S_1, a) = S_2$

$\delta(S_1, b) = S_1$

$\delta(S_2, a) = S_2$

$\delta(S_2, b) = S_1$

Visual Representation:

START

↓

$(S_1) \xrightarrow{a} ((S_2))$

↑ ↖

↓ ↷

| b b a

| _____ |

Legend:

- S_1 : Non-accepting state (single circle)
- S_2 : Accepting state (double circle)
- Arrows show transitions

State Meanings:

- $S_1 = \{1, 2, 3\}$: Haven't seen the required final 'a' yet
- $S_2 = \{1, 2, 3, 4\}$: Have seen the final 'a', can accept

Language Accepted: $(a|b)^*a$

- Any sequence of a's and b's ending with 'a'

9. Second Complete Example: $(a|\epsilon)bc^*\#$

9.1 Setup

Original RE: $(a|\epsilon)bc^*$

Augmented RE: $((a|\epsilon)bc^*)\#$

Position Assignment:

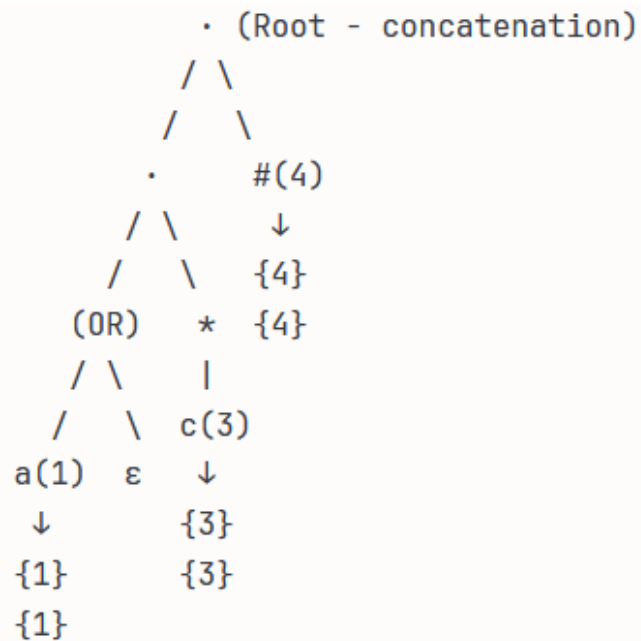
Position 1: a

Position 2: b

Position 3: c

Position 4: #

9.2 Syntax Tree



Explanation:

- Root: concatenation of left part and #
- Left part: concatenation of (a|ε) and bc*
- (a|ε): OR of 'a' and epsilon
- bc*: concatenation of 'b' and c*